



BOOTCAMP FRONTEND & REACT · SEMANA 3

EL DOM

Clase 9 — TechCart deja de ser una foto: catálogo generado, eventos, formulario y carrito que se recuerda

Semana 3 · L·M·V

Proyecto TechCart

STEER
RC
LA

LA SALIDA DEJA DE SER LA CONSOLA Y PASA A SER LA PÁGINA

Agenda

1. Qué es el DOM

el árbol que arma el navegador

2. Seleccionar

querySelector · querySelectorAll

3. Leer y cambiar

textContent · classList · dataset

4. Crear y borrar

el catálogo, desde el arreglo

5. Eventos

burbujeo · delegación · el carrito

6. localStorage

que el carrito sobreviva

7. Formularios dinámicos

submit · preventDefault · validar

8. Remates y cierre

errores frecuentes · 5 tareas



Receso de 15 min después del bloque 4, con el catálogo ya generado

Dos por el chat

¿Por qué la página necesita **servidor local** desde la clase pasada?

¿Qué devuelve `productos.map(...)` cuando lo aplico sobre los seis productos?

EL NAVEGADOR LEE TU HTML Y CON ÉL CONSTRUYE UN ÁRBOL DE OBJETOS

La página también es un árbol

```
document
└─ html
   └─ body
      ├── header - h1 · nav
      ├── main
      │   └─ section#catalogo
      │       └─ div#catalogo-grid
      ├── aside
      └─ footer
```

nodo · elemento

Nodo es cualquier cosa del árbol: etiqueta, texto, comentario. **Elemento** es el nodo que viene de una etiqueta.

document

El objeto que representa el documento entero: la raíz y la puerta de entrada.

El DOM **no es el archivo**. El `index.html` está quieto en el disco. Si cambias el árbol y recargas, vuelve todo al archivo.

PARA CAMBIAR ALGO, PRIMERO HAY QUE ENCONTRARLO

Seleccionar con los **selectores de CSS**

querySelector

El **primero** que coincide, o null si no hay ninguno.

```
document  
.querySelector("h1")
```

querySelectorAll

Todos los que coinciden. Si no hay ninguno, lista vacía, nunca null.

```
document  
.querySelectorAll(".tar  
jeta")
```

getElementById

La forma clásica, anterior a las otras dos. Solo para id, y para un id es la más directa.

```
document  
.getElementById("catalo  
go")
```

DOS COSAS QUE MUERDEN Y QUE CASI NINGÚN TUTORIAL DICE

querySelectorAll no devuelve un arreglo

NodeList

Tiene `length` y `forEach`, pero **no tiene** `map`, `filter` ni `reduce`.

Array.from(lista)

O el spread de la Clase 8: `[...lista]`. Ahí sí valen todos los métodos de la Clase 7.

Y la lista es **estática**: es una foto del momento en que se pidió. Si después aparecen veinte elementos que coinciden con el mismo selector, la lista sigue diciendo lo que decía antes.

LA MISMA ASIGNACIÓN, DOS RESULTADOS DISTINTOS

textContent frente a innerHTML

```
elemento.textContent = "  
<b>Ana</b>"
```

En pantalla se ve: Ana
Lo pone **tal cual**, como texto. No interpreta nada.

```
elemento.innerHTML = "  
<b>Ana</b>"
```

En pantalla se ve: **Ana** en negrita.
El navegador lo **interpreta como etiquetas** y construye elementos de verdad.

Si el texto viene de una persona o de una API, textContent siempre. Un comentario con una etiqueta <script> adentro, metido con innerHTML, se convierte en código corriendo en la página de todos tus visitantes. Eso es **inyección de HTML**, y su versión con JavaScript se llama **XSS**.

ADEMÁS DEL CONTENIDO, DE UN ELEMENTO SE CAMBIAN TRES COSAS MÁS

Clases, atributos y data-

classList

add · remove · contains ·
toggle

```
boton.classList  
  .toggle("hidden",  
  vacio)
```

Con segundo argumento: deja la clase si la condición es verdadera y la quita si es falsa.

getAttribute · setAttribute

getAttribute devuelve lo **escrito** en el HTML; la propiedad del objeto puede devolver otra cosa ya resuelta.

```
<a href="/ayuda">  
a.getAttribute("href")  
// "/ayuda"  
a.href  
// "http://.../ayuda"
```

dataset

Cualquier atributo que empiece con data- es HTML válido y guarda información nuestra en el elemento.

```
<button data-id="4">  
boton.dataset.id  
// "4" ← cadena
```

Siempre devuelve cadena.

SE PUEDE TOCAR CUALQUIER PROPIEDAD DE CSS DESDE JAVASCRIPT, Y CASI NUNCA CORRESPONDE

style: cuándo sí y cuándo no

```
elemento.style.display =  
"none"
```

Queda como estilo en línea y le gana a **toda** la hoja de estilos, incluidos el modo oscuro y los puntos de corte. Después el CSS no lo puede volver a mostrar sin pelear.

```
elemento.classList.toggle("hidden",  
vacio)
```

JavaScript cambia el **estado**; cómo se ve lo decide el CSS. Esta es la forma normal.

style sí corresponde cuando el valor es un número **calculado** que ninguna clase puede traer escrito: el ancho de una barra de progreso, una posición que sigue al mouse, una altura medida en el momento. Hoy tenemos exactamente uno: `barraEnvio.style.width = `${avance}%``.

DOS MANERAS DE METER ELEMENTOS NUEVOS, Y LAS DOS SE USAN

Crear y borrar

ELEMENTO POR ELEMENTO

```
const aviso =  
document.createElement("p");  
aviso.textContent = mensaje;  
aviso.classList.add("font-semibold");  
caja.append(aviso);  
  
aviso.remove();
```

createElement fabrica un objeto suelto que todavía no está en ninguna parte. append lo mete (acepta varios nodos y texto; appendChild, uno solo). remove() se borra a sí mismo.

Es la forma segura cuando el contenido viene de una persona.

DE UN TIRÓN, CON PLANTILLA

```
const html =  
lista.map(tarjetaProducto).join("");  
grid.innerHTML = "";  
grid.insertAdjacentHTML("beforeend",  
html);
```

innerHTML **reemplaza** todo el contenido. insertAdjacentHTML **agrega** sin destruir, y la posición se elige entre "beforebegin", "afterbegin", "beforeend" y "afterend".

Es la forma corta para muchas piezas iguales con datos nuestros.

CADA VEZ QUE TOCAS EL DOM, EL NAVEGADOR RECALCULA Y VUELVE A PINTAR

Un solo **toque**, no uno por vuelta

```
// ✗ seis inserciones, y el navegador rehace la página en cada vuelta
for (const p of productos) {
  grid.innerHTML += tarjetaProducto(p);
}
```

```
// ✔ un solo toque: map arma los textos, join los pega, y recién ahí se inserta
grid.insertAdjacentHTML("beforeend", productos.map(tarjetaProducto).join(""));
```

Y el problema peor del de arriba no es la velocidad: `innerHTML +=` lee todo el HTML, le suma el pedazo nuevo y vuelve a construir todo desde cero. Las tarjetas anteriores se destruyen y se crean otra vez en cada vuelta, con sus listeners.

EL MOMENTO DE LA CLASE: SE BORRAN 40 LÍNEAS DE HTML Y SALEN 6 TARJETAS

El catálogo vive en el dato

ANTES · EL CATÁLOGO VIVÍA EN EL HTML

```
<article class="tarjeta ...">
  <figure>...</figure>
  <h3>MacBook Pro 14</h3>
  <p><strong>$1,999.99</strong></p>
  <button>Agregar al carrito</button>
</article>
// ... y lo mismo tres veces más
```

4 productos, escritos a mano. Agregar uno es copiar veinte líneas.

DESPUÉS · EL CATÁLOGO VIVE EN EL ARREGLO

```
<div id="catalogo-grid"></div>

const pintarCatalogo = (lista = productos)
=> {
  const html =
    lista.map(tarjetaProducto).join("");
  grid.innerHTML = "";
  grid.insertAdjacentHTML("beforeend",
    html);
};
```

6 productos, y mañana 200. Agregar uno es agregar un objeto al arreglo.

LA FUNCIÓN NO CORRE AHORA: QUEDA REGISTRADA, ESPERANDO

addEventListener

```
boton.addEventListener("click", (evento) => { ... });
```

↑ a QUIÉN ↑ QUÉ escucho ↑ QUÉ hago

evento.type

Qué pasó: "click".

evento.target

El elemento **exacto** donde ocurrió, que no siempre es al que le puse el listener.

evento.preventDefault()

Cancela lo que el navegador iba a hacer por su cuenta.

Los cuatro que más vas a usar: **click** para botones, **input** para cuando alguien escribe, **change** para selects y checkboxes, **submit** para formularios. Un listener se quita con `removeEventListener`, y hay que pasarle **la misma función**, así que tiene que estar en una variable.

EL EVENTO NO SE QUEDA DONDE NACIÓ

El burbujeo

```
document      ↑
└─ body      ↑
   └─ section ↑
      └─ div#catalogo-grid ↑
         └─ article ↑
            └─ button ← aquí nace
```

sube padre por padre

Nace en el botón y va subiendo por el árbol hasta document. Se llama burbujeo porque sube como una burbuja.

consecuencia directa

Puedo escuchar el clic en **cualquier ancestro** del botón y me entero igual. Con eso se resuelve el problema de la lámina siguiente.

LOS ELEMENTOS QUE JAVASCRIPT CREA NACEN SIN LISTENER

Delegación de eventos

```
// ❌ un listener por botón: funciona hasta el primer repintado
document.querySelectorAll("button").forEach((b) => b.addEventListener("click", ...));
```

```
// ✅ UN listener, en el contenedor, que nunca se reemplaza
grillaCatalogo.addEventListener("click", (evento) => {
  const boton = evento.target.closest("button[data-accion='agregar']");
  if (!boton) return; // el clic cayó en otro lado
  ...
});
```

Cuando escribo en el contenedor, los elementos que había **dejan de existir** y nacen otros, idénticos pero distintos. Es el bug que se reporta siempre con la misma frase: **"el botón funciona la primera vez y después no"**. `evento.target` es dónde nació el clic; `closest` sube buscando el ancestro que coincide, o devuelve `null`.

SI TE LLEVAS UNA COSA DE HOY, QUE SEA ESTA

Un dato, una función que pinta



El evento **nunca** toca la pantalla, y la función que pinta **nunca** cambia el dato. Lo contrario, que cada evento vaya tocando pedacitos por su cuenta, es como se escriben las aplicaciones **desincronizadas**: el contador dice 3, la lista muestra 2 y el total no coincide con ninguno.

🔗 UN MAP<STRING, STRING> QUE SOBREVIVE AL CIERRE DEL PROGRAMA

localStorage guarda solo TEXTO

setItem(clave, valor)

Guarda. El valor tiene que ser una **cadena**.

getItem(clave)

Lee, y devuelve null si esa clave no existe.

removeItem(clave)

Borra esa clave.

```
localStorage.setItem("carrito", [{ id: 1 }, { id: 2 }]);  
localStorage.getItem("carrito");  
// → "[object Object],[object Object]" ← los datos no están
```

Dos propiedades que hay que saber: es **por origen** (protocolo + dominio + puerto), así que solo lo lee una página del mismo sitio; y **persiste**: no se borra al cerrar la pestaña, ni el navegador, ni al apagar la máquina.

DOS FUNCIONES QUE VAN SIEMPRE JUNTAS, Y UN ENVOLTORIO QUE NO ES OPCIONAL

JSON.stringify • JSON.parse • try/catch

```
export const guardarCarrito = (carrito) => {  
  try { localStorage.setItem(CLAVE_CARRITO, JSON.stringify(carrito)); }  
  catch (error) { console.warn("No se pudo guardar el carrito:", error.message); }  
};  
  
export const leerCarrito = () => {  
  try {  
    const crudo = localStorage.getItem(CLAVE_CARRITO);  
    return crudo ? JSON.parse(crudo) : [];  
  } catch (error) { return []; }  
};
```

El try/catch no es prolijidad. `setItem` lanza excepción cuando el almacenamiento está lleno (unos 5 MB por sitio) y en algunos navegadores en modo privado. `JSON.parse` lanza excepción cuando lo guardado quedó a medias o lo escribió una versión anterior. Y eso pasa **al arrancar**: sin el catch, esa persona ve una pantalla en blanco y en tu máquina nunca lo vas a poder reproducir.

LA MISMA API EXACTA, UNA SOLA DIFERENCIA

localStorage frente a sessionStorage

localStorage

Persiste hasta que alguien lo borre. Para lo que debe quedar: el carrito, el tema oscuro, el idioma.

sessionStorage

Se borra **al cerrar la pestaña**. Para lo que solo tiene sentido en esta visita: un formulario largo a medias, un filtro temporal.

Nunca contraseñas, tarjetas ni datos sensibles. Cualquier JavaScript que corra en la página lo puede leer, y la persona también, en la pestaña Application que acabamos de abrir.

LO PRIMERO QUE HACE EL LISTENER, ANTES QUE NADA

submit y preventDefault

```
formulario.addEventListener("submit", (evento) => {  
    evento.preventDefault();  
    ...  
});
```

sin preventDefault

El navegador **envía el formulario y recarga la página**, con los datos en la dirección. Todo lo que estaba en memoria se pierde de vista. Y no hay ningún error: hizo lo que siempre hace.

con preventDefault

La página se queda donde está, el carrito intacto, y **decidimos nosotros** qué mostrar.

El submit se escucha en el **formulario**, no en el botón. Un formulario también se envía apretando Enter en un campo: si escuchas el clic del botón, se te escapa el Enter.

VALIDAR CON JAVASCRIPT NO ES TIRAR LAS VALIDACIONES DE LA CLASE 1

Validar y leer los campos

novalidate

No borra ninguna regla. Apaga **solo** el aviso automático al enviar. `required`, `type`, `pattern`, `minlength`, `min` y `max` siguen vivos.

checkValidity() · reportValidity()

`checkValidity` devuelve `true` o `false` sin mostrar nada.
`reportValidity` devuelve lo mismo **y muestra los globos nativos**, los de la Clase 1.

FormData · elements

```
new FormData(form)
  .get("nombre")

form.elements
  .cantidad.value
```

Uno junta todo el formulario por el atributo `name`; el otro llega a un campo suelto.

POR QUÉ EN TODA ESTA CLASE NO ESCRIBIMOS NI UN `DOMContentLoaded`

Cuándo corre tu **script**

`<script src="main.js">` en el `<head>`

Corre **antes** de que exista el HTML: `querySelector` devuelve `null` y todo falla.

`<script defer>` · o envolver en `DOMContentLoaded`

Corre con el árbol ya armado. Es lo que hacen todos los tutoriales.

`<script type="module">`

Ya viene diferido de fábrica. Es la tercera de las tres cosas que hace `type="module"`, y la usamos desde la Clase 8 sin saberlo.

Hay tres formas de resolver lo mismo: poner el `<script>` al final del `<body>`, ponerle `defer`, o usar `type="module"`. Nosotros usamos la tercera desde la clase pasada, así que el problema estaba resuelto antes de que lo tuviéramos.

CUANDO EL HTML LO ESCRIBE UNA PLANTILLA, SE ESCRIBE UNA VEZ Y SE REPITE 200

Accesibilidad de lo que genera JavaScript

botones

Seis botones que solo dicen "Agregar al carrito" son seis frases iguales para quien no ve la pantalla.

```
aria-label="Agregar  
${nombre} al carrito"
```

imágenes

La foto generada lleva su alt, que sale del dato. Si la pieza es decorativa y no informa nada, aria-hidden="true".

```
alt="${alt}" · aria-  
hidden="true"
```

lo que cambia solo

El contador y el aviso del pedido cambian sin que la persona toque nada. Por eso su zona se anuncia.

```
role="status" aria-  
live="polite"
```

LOS SIETE TROPIEZOS DEL TEMA, CON EL MENSAJE EXACTO QUE VAS A VER

Errores frecuentes

Cannot read properties of null (reading 'textContent')

1. Buscar un elemento antes de que exista, o un typo en el selector. `querySelector` devuelve `null` sin quejarse.

(ningún error: un agujero)

2. `innerHTML` con datos de una persona. Si es texto, `textContent`.

Cannot read properties of null (reading 'addEventListener')

3. Ponerle el listener a un elemento que todavía no se creó. La respuesta es la **delegación**.

(la página parpadea y la URL se llena de ?)

4. Olvidar `preventDefault`: el formulario se envía y recarga.

"[object Object],[object Object]"

5. Guardar un objeto en `localStorage` sin `JSON.stringify`.

"2" + 1 === "21"

6. Tratar como número el `value` de un `input` o un `dataset`: los dos son **texto**.

`document.querySelectorAll(...).map` is not a function

7. Usar la `NodeList` como si fuera un arreglo. `Array.from(...)` o `[...lista]`.

HOY TECHCART DEJÓ DE SER UNA FOTO

Resumen

el DOM

Un árbol de objetos que arma el navegador. No es el archivo.

querySelector(All)

Con los selectores de CSS de la Clase 2.

NodeList

No es arreglo y es estática. `Array.from`.

textContent / innerHTML

Texto frente a etiquetas. Lo de afuera, siempre texto.

classList · dataset

Clases y datos pegados al elemento. `dataset` da cadena.

style

Solo para valores calculados. Lo normal es cambiar clases.

createElement · append · remove

Para contenido que viene de una persona.

plantilla + insertAdjacentHTML

Para muchas piezas iguales. **Un solo toque al DOM.**

el catálogo generado

Vive en el arreglo, no en el HTML. 6 hoy, 200 mañana.

addEventListener

A quién, qué, y qué hago. `target` y `preventDefault`.

burbujeo + delegación

Un listener en el contenedor, y `closest` pregunta quién fue.

checkValidity · reportValidity

Validar con JS sin perder lo nativo de la Clase 1.

FormData · elements

Las dos formas de leer los campos.

localStorage

Solo texto: `JSON.stringify` y `parse`, dentro de `try/catch`.

un dato, una función que pinta

El evento cambia el dato y llama a pintar. Nunca al revés.

LA DIFÍCIL 2 LA VAMOS A NECESITAR ARMADA PARA LO QUE VIENE

5 tareas

FÁCIL

Replicar la clase y sumar el contador en la pestaña: `document.title` con un template literal, llamado desde `pintarCarrito`.

**INTERMEDIA
1**

Filtrar por categoría: una fila de botones con `data-categoria`, un solo listener con delegación, y `pintarCatalogo(lista)`. Comprobar que los botones "Agregar" siguen funcionando después de filtrar.

**INTERMEDIA
2**

Aviso de "sin resultados" cuando la lista a pintar esté vacía, creado con `createElement` y `textContent`, y quitado con `remove()`.

DIFÍCIL 1

Tema oscuro que se recuerda: `classList.toggle("dark")` sobre `documentElement` y la elección guardada en `localStorage`, con su `try/catch`.

DIFÍCIL 2

Carrito con cantidades: cada producto entra una vez con su cantidad, con `find` y `spread` dentro de un `map`, sin mutar nada. Y quitar vuelve a ser **por id**.

Clase 10 · Semana 4

Los productos de verdad

Seis productos los escribí yo a mano. En una tienda real viven en un **servidor** y se piden por internet, con **fetch**. Y ahí aparece algo que cambia cómo se escribe JavaScript: los datos **no llegan al instante**.

¿Y si los productos **no los escribimos nosotros**?

Promesas · **async / await** · manejo de errores

Trae la tienda de hoy funcionando. Y si quieres espiar: abre dummyjson.com/products